



Memory Management

Type	Lecture
Date	@December 27, 2021
Lecture #	3
Lecture URL	https://youtu.be/b4nsGWXNm2c
Notion URL	https://21f1003586.notion.site/Memory-Management-bf48fedf690e4df5991b9b7e9b9e1480
Week #	1

Keeping track of variables

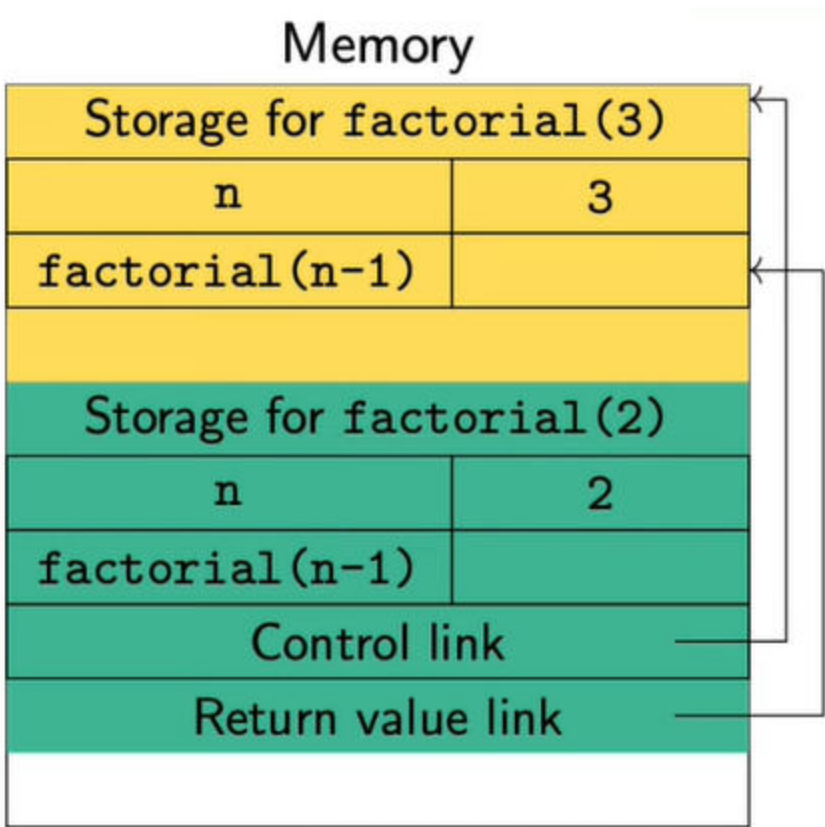
- Variables store intermediate values during computation
 - Typically these are local to a function
 - Can also refer to global variables outside the function
 - Dynamically created data, like nodes in a list
- Scope of a variable
 - When the variable is available for use
 - In the following code, the `x` in `f()` is not in scope withing call to `g()`

```
def f(l):  
    ...  
    for x in l:  
        y = y + g(x)  
    ...  
  
def g(m):  
    ...  
    for x in range(m):  
        ...
```

- Lifetime of a variable
 - How long the storage remains allocated
 - Above, the lifetime of `x` in `f()` is till `f()` exists
 - “Hole in the scope” — variable is alive but not in scope

Memory stack

- Each function needs storage for local variables
- Create **activation record** when function is called
- Activation record are stacked
 - Popped when function exits
 - **Control link** points to start of previous record
 - **Return value link** tells where to store result



Call `factorial(3)`
`factorial(3)` calls `factorial(2)`

- Scope of a variable
 - Variable in activation record at top of stack
 - Access global variables by following control links
- Lifetime of a variable
 - Storage allocated is still on the stack

Passing arguments to a function

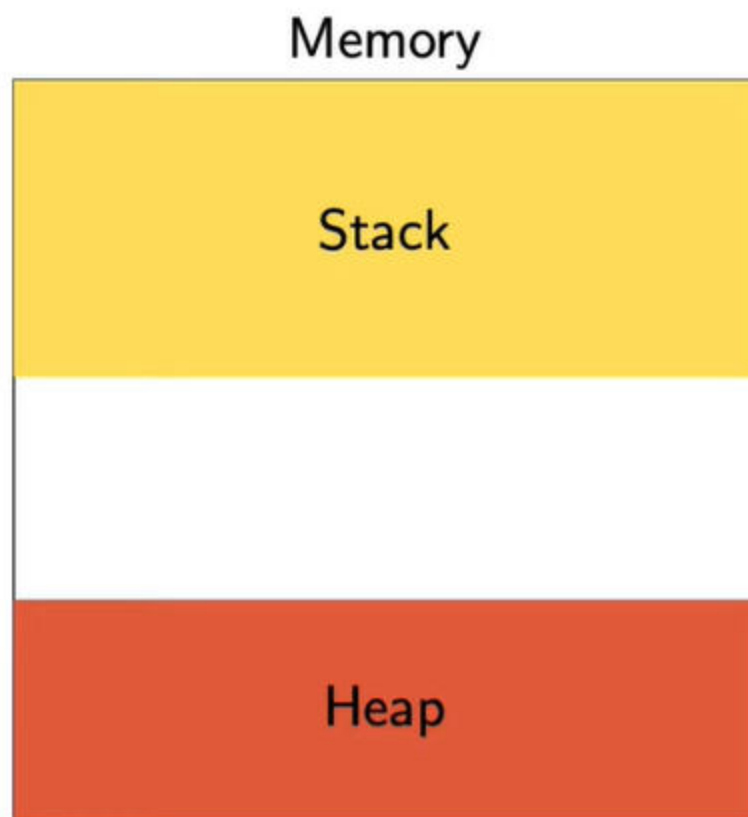
- When a function is called, arguments are substituted for formal parameters

```
def f(a,l):
    ...
    ...
    x = 7
    myl = [8,9,10]
    f(x,myl)
    a = x
    l = myl
    ... code for f() ...
```

- Parameters are part of the activation record of the function
 - Values are populated on function call
 - Like having implicit assignment statements at the start of the function
- Two ways to initialize the parameters
 - **Call by value** → copy the value
 - Updating the value inside the function has not side effect
 - **Call by reference** → parameter points to same location as the argument
 - Can have side-effects
 - It can update the contents, but cannot change the reference itself

Heap

- Function that inserts a value in a linked list
 - Storage for new node allocated inside function
 - Node should persist after function exits
 - Cannot be allocated within activation record
- We need a separate storage for the persistent data
 - Dynamically allocated vs statically declared
 - Usually called the heap
 - Not the same as heap data structure
 - Conceptually, allocate heap storage from “opposite” end with respect to the stack



- Heap store outlives activation record
 - Access through some variable that is in scope

Managing heap storage

- On the stack, variables are deallocated when a function exits
- How do we return unused storage on the heap?
 - After deleting a node in a linked list, deleted node is now dead storage, unreachable
- Manual memory management
 - Programmer explicitly requests and returns heap storage
 - `p = malloc(...)` and `free(p)` in C language
 - Error-prone — memory leaks, invalid assignments
- Automatic garbage collection (Java, Python, ...)
 - Run-time environment checks and cleans up dead storage
 - Mark all storage that is reachable from program variables
 - Return all unmarked memory cells to free space
 - Convenience for programmer vs performance penalty

Summary

- Variables have **scope** and **lifetime**
 - Scope → whether the variable is available in the program

- Lifetime → whether the storage is still allocated
- Activation records for functions are maintained as a stack
 - Control link points to previous activation record
 - Return value link tells where to store results
- Heap is used to store dynamically allocated data
 - Outlives activation record of function that created the storage
 - Need to be careful about deallocating heap storage
 - Explicit deallocation vs automatic garbage collection